

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №5

Решение дифференциальных уравнений

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Определение квадратурных формул	2
2.1.2	Степень точности квадратурной формулы	2
2.1.3	Формула трапеций	2
2.1.4	Обработка точек разрыва	3
2.2	Блок-схема численного метода	4
2.3	Листинг численного метода	7
2.4	Результаты работы программы	14
3	Заключение	16

1 Постановка задач

Дано обыкновенное дифференциальное уравнение первого порядка с начальным условием вида:

$$y' = f(x, y), \quad y(a) = y_0$$

Необходимо найти приближённое решение задачи Коши на интервале $[a, b]$ с заданным параметром точности ε — максимальной допустимой разницей между значениями $y(b)$, которые получены при последовательном уточнении шага интегрирования.

В варианте №1 вычисление осуществляется методом Эйлера. При реализации алгоритма необходимо учитывать следующие условия:

- правая часть уравнения $f(x, y)$ выбирается по номеру из набора подготовленных дифференциальных уравнений;
- шаг интегрирования h должен определяться и автоматически пересчитываться до тех пор, пока не будет выполнено условие точности $|y_{h/2}(b) - y_h(b)| \leq \varepsilon$;
- на графиках должны быть отображены начальное условие (a, y_0) , численное решение, полученное методом Эйлера, и аналитическое решение (при его наличии) для визуального сравнения и оценки накопления погрешности;
- в ходе анализа необходимо исследовать устойчивость метода, а также рассмотреть сценарии практического применения метода.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка итерационного алгоритма решения задачи Коши методом Эйлера с автоматическим подбором шага h и контролем точности по заданному параметру ε .
2. Реализация механизма выбора функции $f(x, y)$ по индексу, построения графиков численного и аналитического решений, а также визуализации начальных условий.
3. Организация ввода данных в формате $a \ b \ y_0 \ f_n \ \varepsilon$, формирование вывода в виде приближённого значения $y(b)$ и проведение анализа устойчивости и погрешности полученного решения.

2 Выполнение работы

2.1 Описание метода

2.1.1 Определение квадратурных формул

Рассмотрим задачу численного вычисления определённого интеграла от функции $f(x)$ на отрезке $[a, b]$:

$$I = \int_a^b f(x) dx.$$

Квадратурной формулой (или *аппроксимирующей квадратурой*) называется выражение вида

$$Q_n[f] = \sum_{i=0}^n w_i f(x_i),$$

где $\{x_i\}_{i=0}^n \subset [a, b]$ — узлы квадратуры, а $\{w_i\}_{i=0}^n$ — весовые коэффициенты. Квадратурная формула аппроксимирует интеграл I конечной линейной комбинацией значений подынтегральной функции.

Погрешность квадратуры определяется как разность следующего вида:

$$E_n[f] = I - Q_n[f]$$

2.1.2 Степень точности квадратурной формулы

Алгебраической степенью точности квадратурной формулы называется наибольшее целое неотрицательное число m , такое что формула точна для всех алгебраических многочленов степени не выше m :

$$\forall k = \overline{0, m} \quad E_n[x^k] = 0, \quad \text{но} \quad E_n[x^{m+1}] \neq 0$$

Иными словами, если подынтегральная функция является многочленом $P_m(x)$ степени m , то квадратурная формула даёт точное значение интеграла.

2.1.3 Формула трапеций

Рассмотрим отрезок $[a, b]$ и построим интерполяционный многочлен Лагранжа первой степени на узлах $\{a, b\}$:

$$L_1(x) = \frac{x-b}{a-b} f(a) + \frac{x-a}{b-a} f(b)$$

Определим остаточный член $R_1(x)$ согласно теореме об остаточном члене интерполяционной формулы Лагранжа:

$$f(x) = L_1(x) + R_1(x), \quad R_1(x) = \frac{f''(\xi(x))}{2} (x-a)(x-b), \quad \xi(x) \in (a, b)$$

Проинтегрируем обе части равенства по отрезку $[a, b]$:

$$\int_a^b f(x) dx = \int_a^b L_1(x) dx + \int_a^b R_1(x) dx$$

Вычислим интеграл от интерполяционного многочлена почленно:

$$\begin{aligned}
 \int_a^b L_1(x) dx &= f(a) \int_a^b \frac{x-b}{a-b} dx + f(b) \int_a^b \frac{x-a}{b-a} dx \\
 &= \frac{f(a)}{a-b} \left[\frac{(x-b)^2}{2} \right]_a^b + \frac{f(b)}{b-a} \left[\frac{(x-a)^2}{2} \right]_a^b \\
 &= \frac{f(a)}{a-b} \left(0 - \frac{(a-b)^2}{2} \right) + \frac{f(b)}{b-a} \left(\frac{(b-a)^2}{2} - 0 \right) \\
 &= \frac{b-a}{2} f(a) + \frac{b-a}{2} f(b) = \frac{b-a}{2} (f(a) + f(b))
 \end{aligned}$$

Для оценки погрешности проинтегрируем остаточный член. Поскольку $(x-a)(x-b) \leq 0$ на $[a, b]$, а f'' непрерывна, по интегральной теореме о среднем:

$$\int_a^b \frac{f''(\xi(x))}{2} (x-a)(x-b) dx = \frac{f''(\xi)}{2} \int_a^b (x-a)(x-b) dx, \quad \xi \in (a, b)$$

Вычислим интеграл от квадратного многочлена:

$$\int_a^b (x-a)(x-b) dx = \int_a^b (x^2 - (a+b)x + ab) dx = -\frac{(b-a)^3}{6}$$

Подставляя результат, получаем остаточный член формулы трапеций:

$$\int_a^b R_1(x) dx = -\frac{(b-a)^3}{12} f''(\xi)$$

Окончательно, полная формула трапеций записывается в виде:

$$\int_a^b f(x) dx = \frac{f(a) + f(b)}{2} \cdot h - \frac{h^3}{12} f''(\xi), \quad \xi \in (a, b), \quad h = b - a$$

Можно заметить, что алгебраическая степень точности метода трапеций равна $m = 1$. Это следует из погрешности квадратуры: если f — многочлен первой степени, то $R_1(x) \equiv 0$.

2.1.4 Обработка точек разрыва

Если функция $f(x)$ имеет разрывы на отрезке интегрирования, квадратурная формула может давать некорректный результат или приводить к исключительным ситуациям (*деление на ноль, переполнение*).

1. **Устранимый разрыв или скачок.** Интеграл Римана существует, и алгоритм должен заменить значение $f(c)$ на одно из значений $f(c - \varepsilon(h))$, $f(c + \varepsilon(h))$, где $\varepsilon(h)$ — некоторая малая величина.
2. **Бесконечный разрыв.** Интеграл Римана не существует, и алгоритм должен прервать вычисления, уведомив пользователя об ошибке.

Таким образом, в реализации метода трапеций мы должны проанализировать подынтегральную функцию на наличие особенностей и адаптировать разбиение интервала интегрирования в зависимости от типа разрыва.

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- f_n — номер подынтегральной функции; целое число.
- a, b — границы интервала интегрирования; вещественные числа двойной точности.
- ε — точность вычислений, то есть максимально допустимая разность между двумя последовательными итерациями; вещественное число двойной точности.
- M — максимально допустимый размер разбиения; целое число.
- $\mathcal{D}$ — метаданные о расположении и типах разрывов функции (*точки первого/второго рода, интервалы неопределённости*).

На выходе метода ожидается вещественное число двойной точности — вычисленное значение интеграла.

Остановка итерационного процесса происходит при достижении заданной точности ε ($|I_k - I_{k-1}| < \varepsilon$) или при превышении лимита разбиений M .

Далее приводятся блок-схемы, реализующие численный метод.

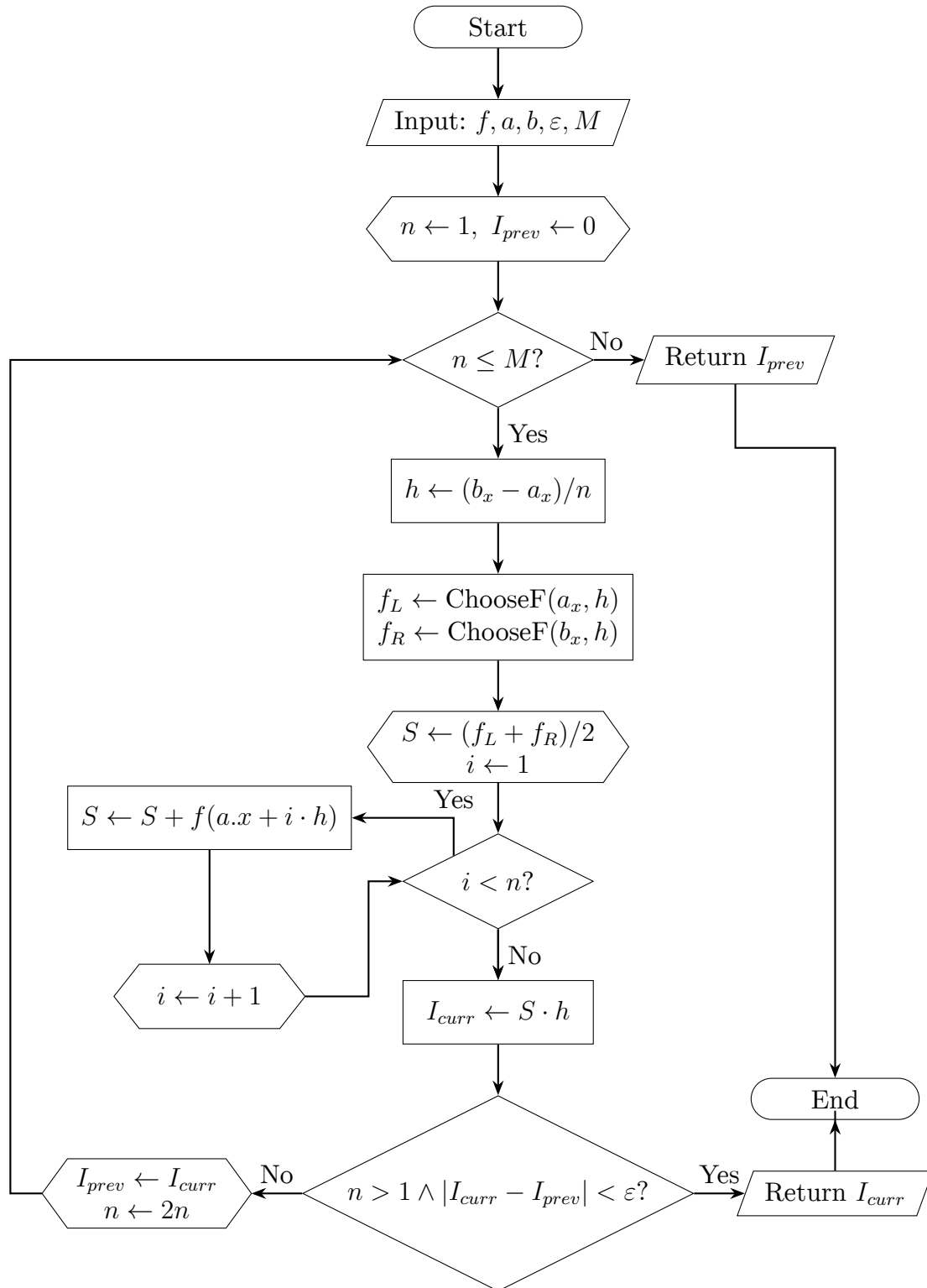


Схема 1 — Блок-схема безопасного вычисления формулы трапеций.

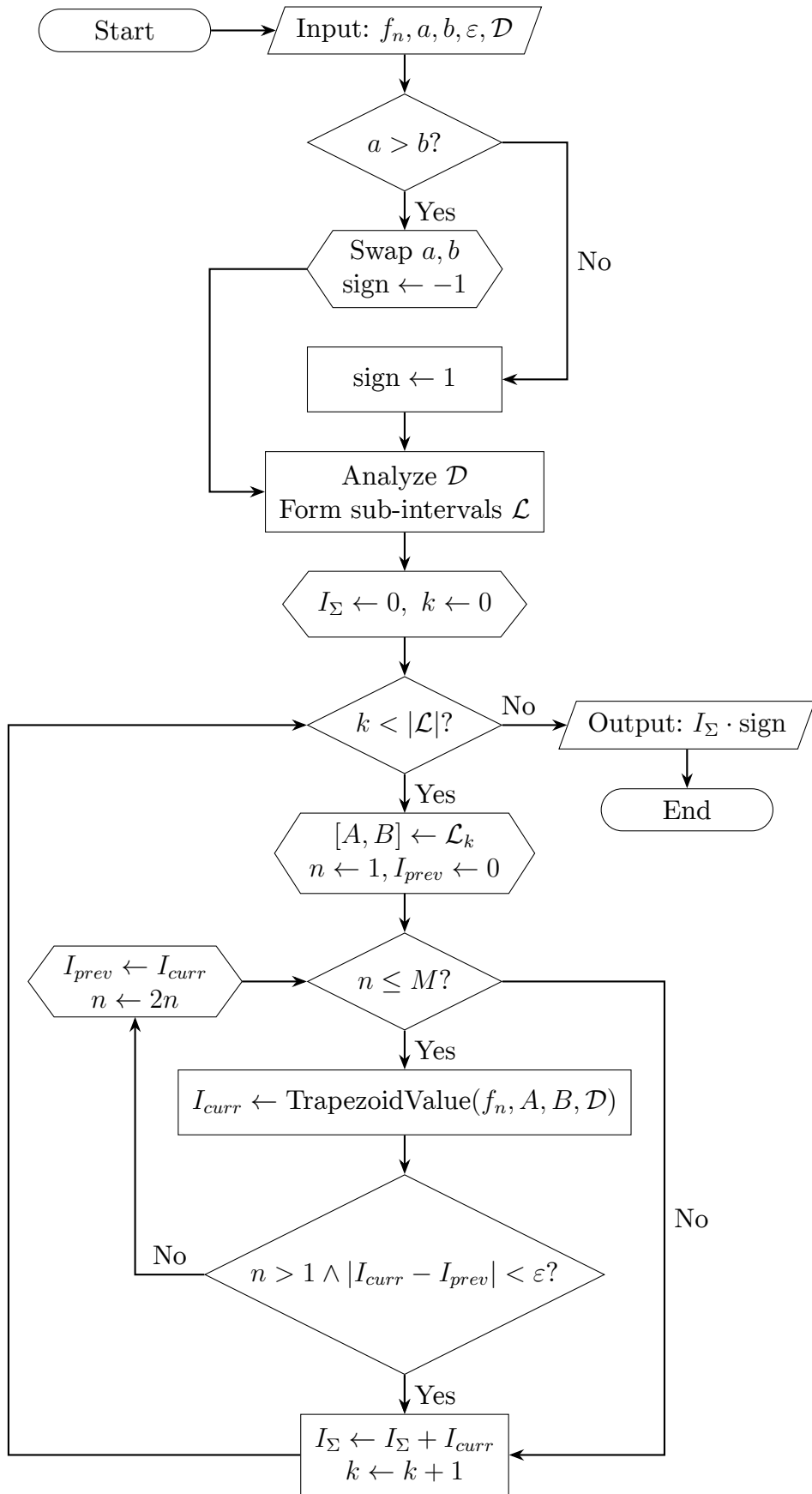


Схема 2 — Блок-схема адаптивного метода трапеций.

2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```
1 #ifndef INTEGRATION_H
2 #define INTEGRATION_H
3
4 #include <limits>
5
6 #include "preamble.h"
7 #include "unordered_map"
8 #include "vector"
9
10 struct DiscontinuityDot {
11     double x;
12     // 0 - continuous
13     // 1 - first order
14     // 2 - second order
15     std::size_t order;
16 };
17
18 struct DiscontinuityPlace {
19     bool is_interval;
20     DiscontinuityDot left;
21     DiscontinuityDot right;
22 };
23
24 struct FunctionMeta {
25     std::function<double(double)> f;
26     std::function<double(double, double)> integrate;
27     std::vector<DiscontinuityPlace> discontinuity_places;
28 };
29
30 struct IntegrationResult {
31     double result;
32     double exact_result;
33     bool has_result;
34     bool has_discontinuity;
35     std::string message;
36     std::size_t num_iterations;
37 };
38
39 class IntegrateFunctions {
40 public:
41     IntegrateFunctions(double epsilon)
42         : IntegrateFunctions(epsilon, 1e6, std::unordered_map<std::
43             ::size_t, FunctionMeta>{}) {}
44
45     IntegrateFunctions(
46         double epsilon,
47         std::size_t max_iterations,
```

```

48     std::unordered_map<std::size_t, FunctionMeta> funcs
49 )
50     : _funcs(funcs), _epsilon(epsilon), _max_iterations(
51         max_iterations) {
52
53 void AddFunction(const std::size_t number, const FunctionMeta&
54     func) {
55     this->_funcs[number] = func;
56 }
57
58 IntegrationResult Integrate(std::size_t number, double a,
59     double b) {
60     double result_sign = 1.0;
61     if (b < a) {
62         std::swap(a, b);
63         result_sign = -1.0;
64     }
65     if (this->_funcs.count(number) == 0U) {
66         throw std::invalid_argument("no integration functions
67             registered under specified number");
68     }
69     double result = 0.0;
70     FunctionMeta meta = this->_funcs[number];
71
72     auto get_point_order = [&](double x) -> std::size_t {
73         for (const auto& place : meta.discontinuity_places) {
74             if (place.is_interval) {
75                 if (x > place.left.x && x < place.right.x) {
76                     return 2;
77                 }
78             } else {
79                 if (std::abs(place.left.x - x) < 1e-12) {
80                     return place.left.order;
81                 }
82             }
83         }
84         return 0;
85     };
86
87     std::size_t order_a = get_point_order(a);
88     std::size_t order_b = get_point_order(b);
89
90     std::vector<DiscontinuityDot> split_points = {
91         {a, order_a}
92     };
93
94     bool is_discontinued = order_a != 0 || order_b != 0;
95
96     for (const auto& place : meta.discontinuity_places) {
97         if (place.is_interval) {

```

```

95     if (place.left.x < b && a < place.right.x) {
96         return {
97             0.0,
98             0.0,
99             false,
100             true,
101             "Integrated function has discontinuity or does not
              defined in current interval",
102             0,
103         };
104     }
105     } else {
106         double dx = place.left.x;
107         if (dx >= a && dx <= b) {
108             if (place.left.order == 2) {
109                 return {
110                     0.0,
111                     0.0,
112                     false,
113                     true,
114                     "Integrated function has discontinuity or does
                        not defined in current interval",
115                     0,
116                 };
117             }
118             if (dx != a && dx != b) {
119                 is_discontinued = true;
120                 split_points.push_back({dx, place.left.order});
121             }
122         }
123     }
124 }
125 split_points.push_back({b, order_b});
126
127 double total_result = 0.0;
128 std::size_t intervals = 0;
129 for (std::size_t i = 0; i < split_points.size() - 1; ++i) {
130     DiscontinuityDot sub_a = split_points[i];
131     DiscontinuityDot sub_b = split_points[i + 1];
132     SubintervalResult sub_res = _IntegrateSubInterval(meta.f,
        sub_a, sub_b);
133     total_result += sub_res.integral;
134     intervals += sub_res.num_iterations;
135 }
136
137 return {
138     total_result * result_sign,
139     meta.integrate(a, b),
140     true,
141     is_discontinued,
142     "",

```

```

143         intervals,
144     };
145 }
146
147 private:
148     struct SubintervalResult {
149         double integral;
150         std::size_t num_iterations;
151     };
152
153     SubintervalResult _IntegrateSubInterval(
154         const std::function<double(double)>& f, const
155         DiscontinuityDot& a, const DiscontinuityDot& b
156     ) {
157         auto safe_eval = [&](double x, double h_step, bool is_left,
158             bool is_continuous) -> double {
159             if (!is_continuous) {
160                 double eps = std::max(1e-12, h_step * 0.01);
161                 if (is_left) {
162                     double v_r = f(x + eps);
163                     bool bad_r = std::isnan(v_r) || std::isinf(v_r);
164                     if (!bad_r) {
165                         return v_r;
166                     }
167                 }
168
169                 double v_l = f(x - eps);
170                 bool bad_l = std::isnan(v_l) || std::isinf(v_l);
171                 if (!bad_l) {
172                     return v_l;
173                 }
174                 return 0.0;
175             }
176             return f(x);
177         };
178
179         double prev_res = 0.0;
180         std::size_t n = 1;
181
182         std::size_t prev_n = 1;
183         while (n <= this->_max_iterations) {
184             double h = (b.x - a.x) / n;
185             double current_res =
186                 0.5 * (safe_eval(a.x, h, true, a.order == 0) +
187                     safe_eval(b.x, h, false, b.order == 0));
188             for (std::size_t i = 1; i < n; i++) {
189                 current_res += f(a.x + i * h);
190             }
191             current_res *= h;
192
193             if (n > 1 && std::abs(current_res - prev_res) < this->
194                 _epsilon) {

```

```

190     return {current_res, n};
191 }
192 prev_res = current_res;
193 prev_n = n;
194 n *= 2;
195 }
196 return {prev_res, prev_n};
197 }
198
199 std::unordered_map<std::size_t, FunctionMeta> _funcs;
200 const std::size_t _max_iterations;
201 const double _epsilon;
202 };
203
204 IntegrateFunctions InitMain(double epsilon) {
205     IntegrateFunctions integrate = IntegrateFunctions(epsilon);
206     integrate.AddFunction(
207         1,
208         FunctionMeta{
209             get_function(1),
210             [](double a, double b) { return log(std::abs(b / a)); },
211             {
212                 DiscontinuityPlace{
213                     false,
214                     {
215                         0.0,
216                         2,
217                     },
218                 },
219             }
220         );
221     integrate.AddFunction(
222         2,
223         FunctionMeta{
224             get_function(2),
225             [](double a, double b) {
226                 double result = 0.0;
227                 if (b < a) {
228                     std::swap(a, b);
229                 }
230                 if (a < 0) {
231                     result += sineIntegral(-a);
232                     if (b < 0) {
233                         result -= sineIntegral(-b);
234                     } else {
235                         result += sineIntegral(b);
236                     }
237                 } else {
238                     result = sineIntegral(b) - sineIntegral(a);
239                 }

```

```

240         return result;
241     },
242     {
243         DiscontinuityPlace{
244             false,
245             {
246                 0.0,
247                 1,
248             },
249         },
250     }
251 );
252 integrate.AddFunction(
253     3,
254     FunctionMeta{
255         get_function(3),
256         [](double a, double b) { return (pow(b, 3) - pow(a, 3)
257             ) / 3 + 2 * (b - a); },
258     },
259 );
260 integrate.AddFunction(
261     4,
262     FunctionMeta{
263         get_function(4),
264         [](double a, double b) { return pow(b, 2) - pow(a, 2)
265             + 2 * (b - a); },
266     },
267 );
268 integrate.AddFunction(
269     5,
270     FunctionMeta{
271         get_function(5),
272         [](double a, double b) { return b * log(b) - a * log(a)
273             ) + a - b; },
274         {
275             DiscontinuityPlace{
276                 true,
277                 {
278                     -std::numeric_limits<double>::infinity(),
279                     0,
280                 },
281                 {
282                     0.0,
283                     2,
284                 },
285             },
286         },
287 );
288 return integrate;

```

```

288 }
289
290 double calculate_integral(double a, double b, int f, double
    epsilon) {
291     IntegrateFunctions integrate = InitMain(epsilon);
292     IntegrationResult result = integrate.Integrate(static_cast<std
        ::size_t>(f), a, b);
293     has_discontinuity = result.has_discontinuity;
294     if (!result.has_result) {
295         error_message = result.message;
296     }
297     return result.result;
298 }
299
300 #endif

```

Листинг 1 — Файл integration.hpp

```

1  #include <iostream>
2  #include <unordered_map>
3
4  #include "lib/integration.hpp"
5
6  struct TestCase {
7      std::size_t func_number;
8      double a;
9      double b;
10 };
11
12 int main() {
13     IntegrateFunctions obj = InitMain(1e-6);
14
15     std::vector<TestCase> tests = {
16         {1, 0.1, 2.0},
17         {1, -1.0, 1.0},
18         {2, -1.0, 1.0},
19         {2, 0.0, 1.0},
20         {3, 0.0, 3.0},
21         {4, 0.0, 5.0},
22         {5, 1.0, std::numbers::e},
23         {5, -1.0, 1.0},
24     };
25     for (const TestCase& t : tests) {
26         std::cout << "--- Solution for function #" << t.func_number
            << ":\n";
27         IntegrationResult res = obj.Integrate(t.func_number, t.a, t.
            b);
28         if (!res.has_result) {
29             std::cout << "[Error] " << res.message << '\n';
30             continue;
31         }
32         if (res.has_discontinuity) {

```

```

33     std::cout << "[Warning] The function has discontinuity.
        The interval was split down." << '\n';
34 }
35 std::cout << "Bounds: [" << t.a << ", " << t.b << "]\n";
36 std::cout << "Result: " << res.result << '\n';
37 std::cout << "Exact: " << res.exact_result << '\n';
38 std::cout << "Difference: " << std::abs(res.exact_result -
        res.result) << '\n';
39 std::cout << "Intervals: " << res.num_iterations << '\n';
40 }
41 }

```

Листинг 2 — Файл main.cpp

2.4 Результаты работы программы

Рассмотрим пять функций с различными аналитическими особенностями.

1. *Функция с разрывом второго рода:*

$$f_1(x) = \frac{1}{x}, \quad x \in \mathbb{R} \setminus \{0\}.$$

Имеет бесконечный разрыв в точке $x = 0$. Интеграл в смысле Римана не существует при $0 \in [a, b]$. Алгоритм должен вернуть ошибку.

2. *Функция с устранимым разрывом:*

$$f_2(x) = \frac{\sin x}{x}, \quad \lim_{x \rightarrow 0} f_2(x) = 1.$$

В точке $x = 0$ функция не определена, но предел существует. Алгоритм должен корректно вычислить интеграл, используя алгоритмическое среднее или односторонние пределы.

3. *Квадратный многочлен:*

$$f_3(x) = x^2 + 2, \quad x \in \mathbb{R}.$$

Не имеет особенностей.

4. *Линейная функция:*

$$f_4(x) = 2x + 2, \quad x \in \mathbb{R}.$$

Метод трапеций должен дать точное значение интеграла уже при первом разбиении.

5. *Натуральный логарифм:*

$$f_5(x) = \ln x, \quad x > 0.$$

Не определена при $x \leq 0$. При $a \leq 0$ алгоритм должен вернуть ошибку; при $a > 0$ интеграл существует и вычисляется численно.

Ниже приведены результаты интегрирования указанных функций методом трапеций с точностью $\varepsilon = 10^{-5}$ и лимитом разбиений $M = 10^6$.

i	$[a, b]$	I_{exact}	I_{num}	$ I_{\text{num}} - I_{\text{exact}} $	Разбиения	Статус
1	$[0.1, 2.0]$	2.99573	2.99573	$1.1789 \cdot 10^{-7}$	16384	OK
1	$[-1.0, 1.0]$	—	—	—	0	ERR
2	$[-1.0, 1.0]$	1.89217	1.89217	$1.91478 \cdot 10^{-7}$	1024	WARN
2	$[0.0, 1.0]$	0.946083	0.946083	$9.5739 \cdot 10^{-8}$	512	WARN
3	$[0.0, 3.0]$	15	15	$2,68221 \cdot 10^{-7}$	1	OK
4	$[0.0, 5.0]$	35	35	0	1	OK
5	$[1.0, 2.71828]$	1	1	$1.48323 \cdot 10^{-7}$	1024	OK
5	$[-1.0, 1.0]$	—	—	—	0	ERR

Таблица 1 — Сводная таблица результатов тестирования метода трапеций.

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

Почему степень точности квадратурной формулы определяется через многочлены? По аппроксимационной теореме Вейерштрасса, любую непрерывную на отрезке функцию можно сколь угодно точно приблизить алгебраическим многочленом. Следовательно, если квадратурная формула точно интегрирует многочлены высокой степени, то для гладких функций её погрешность будет определяться лишь погрешностью аппроксимации.

То есть степень точности описывает порядок малости погрешности метода при стремлении шага разбиения к нулю.

Какая асимптотическая сложность у алгоритма? Как видно из блок-схемы, вычислительная сложность одной итерации метода трапеций *линейная* относительно количества разбиений отрезка. Так как алгоритм последовательно удваивает мощность сетки, а погрешность метода убывает как $O(h^2)$, то общее число вычислений функции до достижения точности ε оценивается как $O(\varepsilon^{-1/2})$ (без учёта верхнего предела по мощности разбиения).

Пространственная сложность алгоритма в идеале равна $O(1)$: в памяти могут храниться лишь переменные для аккумуляции суммы, границы отрезков и текущий шаг. Остальные структуры данных, которые используются в реализации, нужны преимущественно для тестирования алгоритма.

В чём преимущество метода трапеций? Главное преимущество метода в его простоте: для вычисления интеграла достаточно лишь последовательно суммировать значения функции в узлах сетки. Методу не нужно вычислять производные или решать дополнительные системы. Ещё можно добавить, что метод трапеций точно интегрирует линейные функции при одном разбиении.

Какие недостатки обнаружены у метода трапеций? Основной недостаток — низкий порядок аппроксимации ($O(h^2)$). Для достижения высокой точности требуется серьёзное увеличение числа разбиений, что ведёт к росту вычислительных затрат и накоплению ошибок округления.

Метод также неэффективен для функций, у которых производная быстро меняется: кусочно-линейная аппроксимация плохо описывает такое поведение и требует сетки большей мощности.

Если сравнивать его с методом Симпсона, метод трапеций требует больше вычислений функции для достижения сопоставимой точности на гладких участках (*у последнего метода алгебраическая степень точности — 3*).

Почему количество разбиений для функций с особенностями существенно возрастает? Метод трапеций основан на кусочно-линейной аппроксимации подынтегральной функции. В окрестности точки разрыва первого рода или устранимой особенности поведение функции резко отличается от линейного, что приводит к локальному росту погрешности аппроксимации.

Для выполнения глобального критерия останова $|I_k - I_{k-1}| < \varepsilon$ алгоритму нужно многократно удваивать число разбиений, пока шаг h не станет достаточно малым, чтобы вклад ошибки в окрестности особенности уменьшился ниже заданного порога.